

 UFRPE	Universidade Federal Rural de Pernambuco Unidade Acadêmica de Belo Jardim Engenharia de Controle e Automação	
	Roteiro da Prática 2 – Acionando uma Sequência de LEDs	
Disciplina:	Microcontroladores e Microprocessadores e Engenharia de Sistemas Embarcados	
Professor:	Henrique Patriota	
Alunos:		
Bancada:		Data:

1. Introdução

Esta prática introduz os registradores de I/O como um ponto de partida para a programação de sistemas embarcados em baixo nível. Como destacado nas aulas anteriores, o ATmega328P é um microcontrolador de 8 bits embarcado na plataforma Arduino UNO. Todos os seus registradores são de 8 bits. Especificamente para os registradores responsáveis pela configuração de seus terminais de entrada e saída (I/O), as configurações de seus bits permitem definir se um terminal será de entrada ou saída e, para os terminais definidos como saídas, permite definir seu nível lógico como alto (1) ou baixo (0).

2. Registradores de I/O: os DDRx e PORTx

Na IDE Arduino, da função usada para definir um terminal digital ou analógico como entrada ou saída é a ***pinMode(número do terminal, INPUT ou OUTPUT)***. Em baixo nível, devemos configurar os bits do registrador de direção de bit DDRx, com x = D, B ou C. De acordo com o datasheet do ATmega328P, os terminais digitais de 0 a 7 compõe o PORTD, os terminais digitais de 8 a 13 compõe o PORTB e os terminais analógicos de A0 a A5 compõe o PORTC. Note que os registradores PORTB e PORTC só usam 6 bits dos 8 bits disponíveis, os bits restantes recebem 0.

A função usada para definir um nível alto ou baixo em um terminal digital, na IDE Arduino, é a ***digitalWrite(número do terminal, nível lógico)***. Em baixo nível, essa função é substituída pelos próprios PORTx, com x = D, B e C. Ou seja, basta definir, de forma conveniente, os bits do PORTD, PORTB e PORTC.

Na Figura 1, tem-se um exemplo de uma aplicação que configura todos os terminais digitais pares como saídas. No loop infinito, ***while(1){ }***, os terminais digitais configurados como saídas recebem nível alto. Como resultado, se tivéssemos LEDs conectados de forma adequada nos terminais pares do microcontrolador, todos estariam acesos. Veja a Figura 2.

CLEAR BIT

```
PORTD&=~(1<<5); //configura em nível baixo no terminal digital 5
```

TOGGLE BIT

```
PORTD^=(1<<5); //comuta o nível lógico no terminal 5, ou seja, troca de 0 para 1 e 1 para 0.
```

TEST BIT

```
PORTD&=(1<<5); //se o nível lógico do terminal 5 for 1, o resultado será 1, caso contrário, 0.
```

4. Descrição da Prática

Na Figura 3 é mostrado o esquema do hardware que deve ser montado. Tratam-se de 8 LEDs conectados nos terminais digitais de 0 a 7 do microcontrolador.

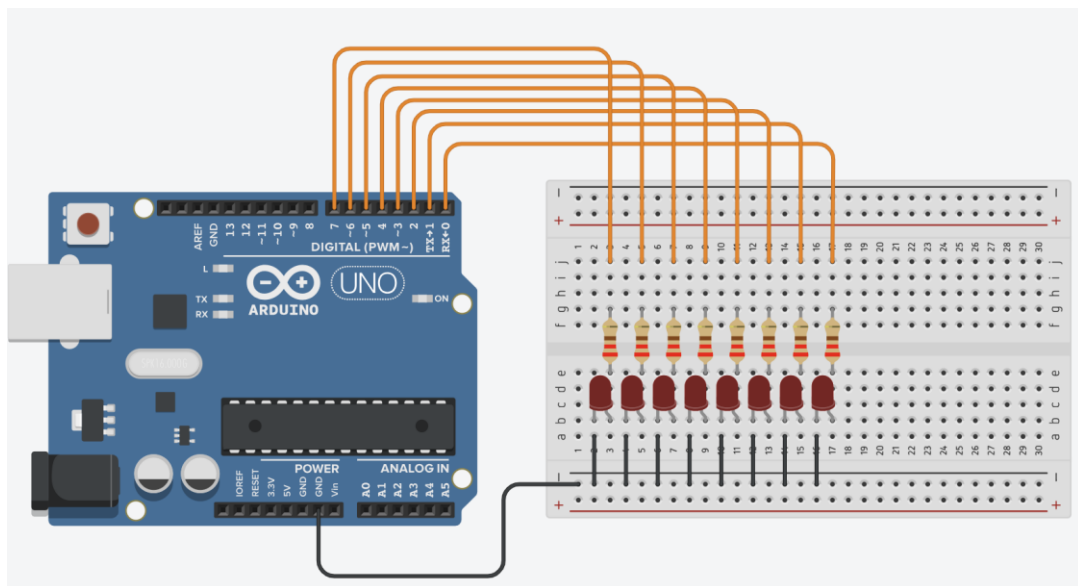


Figura 3 – Esquema de montagem da sequência de LEDs.

Uma rotina de *software* deve implementada para que os LEDs sejam acionados da direita para a esquerda, ou seja, do terminal 0 até o 7, com intervalos de tempo de 500 ms. Quando todos os LEDs estiverem acesos, após 1 s, todos os LEDs devem apagar e manterem esse estado por 1 s. Após isso, a sequência deve repetir a rotina inicial, acionando os LEDs da direita para a esquerda, com intervalos de tempo de 500 ms. **A PROGRAMAÇÃO DEVE SER REALIZADA USANDO OS REGISTRADORES DE I/O.**

DESAFIO

Implemente uma rotina para que apenas um LED seja aceso por vez, da direita para a esquerda. Com isso, quando o LED 7 for aceso, a ordem deve ser invertida, de modo que o acionamento seja

da esquerda para a direita, com um LED sendo aceso por vez, até que o LED 0 seja aceso. Com isso, retorna-se para a rotina original, com os acionamentos da direita para a esquerda.